

COLECCIÓN RUST PÍXEL A PÍXEL

Cuaderno 4 de Rust

Rust Robusto y Conectado

Recomendado a partir de los 18-19 años



Prototipo Editorial

Laboratorio de Código Instantáneo

Índice General

1. Introducción	1
1.1. Qué vas a recordar y aprender en este volumen?	1
1.2. 🌱 Paréntesis Didáctico: Entendiendo los Tipos Genéricos (<T>, <E>)	1
1.2.1. El problema: Repetir código para cada tipo de dato	2
1.2.2. La solución: Las plantillas genéricas (<T>)	2
1.2.3. Funciones con más de un genérico (<T, E>)	2
1.2.4. ¿Cómo se aplica esto a las enumeraciones Option y Result que veremos luego?	2
2. Gestión de Errores Profesional en Rust	4
2.1. Errores recuperables vs. Irrecuperables	4
2.2. El tipo Option<T>	4
2.3. El tipo Result<T, E>	6
2.4. El tipo de dato unidad	7
2.5. El operador de propagación ?	7
3. Interacción con el Sistema Operativo y Archivos	9
3.1. Leyendo archivos de texto	9
3.2. Creación y escritura de datos en archivos	10
4. Gestión con Cargo y el Ecosistema de Rust	13
4.1. ¿Cuáles son las funciones principales de Cargo?	13
4.2. Los comandos esenciales de Cargo	13
4.3. Cargo.toml (El manifiesto del desarrollador)	13
4.4. Cargo.lock (El registro exacto del sistema)	14
4.5. ¿Qué son las crates y Crates.io?	14
5. Procesamiento de Datos y Proyectos Prácticos	15
5.1. Parseo numérico	15
5.2. Proyecto 1: El Gestor de Tareas Pendientes (To-Do-List). Grabar en disco	16
5.3. Proyecto 2: Herramienta CLI para automatización del sistema	20
5.4. Atrapar varios tipos de errores en un único Result	22
5.5. Proyecto 3: Analizador Estadístico de archivos .csv	22
6. Ejercicios resueltos	26
6.1. Enunciados	26
6.1.1. Ejercicio 1: El Validador de Edades (Uso avanzado de Option<T>)	26
6.1.2. Ejercicio 2: Conversión Segura de Temperatura (Uso de Result<T, E>)	26
6.1.3. Ejercicio 3: Copia de Seguridad de Archivos (Manejo del Sistema de Archivos y del operador ?)	26
6.1.4. Ejercicio 4: Calculadora Parser Robusta (Manejo de strings y entradas numéricas)	26
6.1.5. Ejercicio 5: Registro de Visitas con Fecha Simulada (Escritura persistente) .	26
6.2. Soluciones	26
6.2.1. Ejercicio 1: El Validador de Edades (Uso avanzado de Option<T>)	26
6.2.2. Ejercicio 2: Conversión Segura de Temperatura (Uso de Result<T, E>)	27

6.2.3. Ejercicio 3: Copia de Seguridad de Archivos (Manejo del Sistema de Archivos y ?)	28
6.2.4. Ejercicio 4: Calculadora Parser Robusta (Manejo de strings y entradas numéricas)	28
6.2.5. Ejercicio 5: Registro de Visitas con Fecha Simulada (Escritura persistente)	29
7. Ejercicios propuestos	31
7.1. Ejercicio 1: El Control de Altitud (Práctica con Option<T>)	31
7.2. Ejercicio 2: El Validador de Contraseñas del Sistema (Práctica con Result<T, E>)	31
7.3. Ejercicio 3: El Lector Defensivo de Archivos (Manejo de archivos y ?)	31
7.4. Ejercicio 4: El Procesador de Inventario (Parsing y operaciones matemáticas)	32
7.5. Ejercicio 5: El Diario Personal Orientado a Registros (Uso de openOptions)	32

1. Introducción

¡Bienvenido al Cuaderno 4 de tu camino de aprendizaje en Rust! Hasta este momento, en los tres volúmenes anteriores has aprendido a comunicarte con el compilador, a estructurar datos en la memoria y a modelar el mundo real mediante objetos y colecciones. Has tenido también en cuenta que los programas no pueden asumir un entorno «perfecto» en el que el usuario siempre introduce el dato correcto, los archivos siempre existen y los recursos nunca fallaban.

Una de las razones que llevaron a crear el lenguaje Rust fue proporcionar seguridad al código cuando se dan algunos o todos estos imprevistos en el entorno de ejecución de un programa.

No nos vamos a cansar de repetir: en el mundo del software profesional, el entorno ideal no existe. Los discos duros se llenan, las redes se caen, los archivos se corrompen y los usuarios cometen errores constantemente al introducir datos. Un programador amateur escribe código que funciona cuando todo va bien; un desarrollador profesional escribe software robusto que sabe qué hacer cuando todo o casi todo va mal.

1.1. Qué vas a recordar y aprender en este volumen?

Este cuaderno está diseñado para transformar tu forma de programar mediante tres pilares fundamentales:

- **Gestión Defensiva de Errores:** Volverás a descubrir por qué Rust no utiliza el peligroso concepto de «valor nulo» (null) y volverás a usar las herramientas **Option<T>** y **Result<T, E>** para anticiparte y desactivar los fallos antes de que ocurran.
- **Persistencia en el Mundo Real:** Aprenderás a conectar tus programas con el sistema operativo. Tu código dejará de ser efímero: aprenderás a leer y escribir archivos de texto plano para guardar información de forma permanente en el disco.
- **El Ecosistema Cargo:** Romperás los límites del lenguaje estándar introduciéndote en **Crates.io**, el almacén comunitario de Rust, aprendiendo a integrar librerías externas de forma automática para procesar datos complejos como archivos **.csv**.

Al terminar este cuaderno, habrás desarrollado tres proyectos clave (un gestor de tareas persistente, una herramienta de automatización para la terminal y un analizador estadístico). Estarás completamente preparado para enfrentarte al desarrollo de aplicaciones reales y conectadas. ¡Empecemos!

1.2. 🧩 Paréntesis Didáctico: Entendiendo los Tipos Genéricos (<T>, <E>)

Al adentrarnos en las herramientas que describe este cuaderno, verás que aparecen expresiones extrañas como **Option<T>** o **Result<T, E>**. Option y Result son dos enumeraciones definidas en Rust y aunque ya las estudiamos en el Cuaderno 3 las volveremos a estudiar ahora con mayor detalle. Pero, ¿qué significan esas letras mayúsculas entre símbolos de mayor y menor que? Se llaman **Tipos Genéricos**, y son una de las herramientas más potentes para ahorrar código.

1.2.1. El problema: Repetir código para cada tipo de dato

Imagina que queremos programar una función muy simple que sume dos números. Si no existieran los genéricos, nos veríamos obligados a escribir una función diferente para cada tipo de número que usemos en Rust:

```
// Tendríamos que escribir esto...
fn sumar_i32(a: i32, b: i32) -> i32 { a + b }
fn sumar_f32(a: f32, b: f32) -> f32 { a + b }
fn sumar_u64(a: u64, b: u64) -> u64 { a + b }
// ¡Qué pérdida de tiempo y qué código tan repetitivo!
```

1.2.2. La solución: Las plantillas genéricas (<T>)

Para evitar esto, Rust nos permite crear una única función genérica que sustituye a todas las anteriores. Usamos la letra T como un comodín (piensa en T como «Tipo de dato cualquiera»). Es como crear una plantilla o un molde:

```
// Usamos <T> para decirle a Rust:
// Esta función acepta un tipo T cualquiera
fn sumar<T>(a: T, b: T) -> T {
    a + b
}
```

Nota académica: Al usar la función en tu código, por ejemplo, **sumar(5i64, 10i64)**, Rust automáticamente «sustituye» en su mente todas las **T** que hay en la definición de la función genérica sumar por **i64**. Si le pasas dos sumandos f32, las sustituye por f32. ¡Una sola función genérica sirve para todos los tipos de números del mundo siempre y cuando admitan la operación suma entre ellos! No podremos con la función anterior sumar dos String.

1.2.3. Funciones con más de un genérico (<T, E>)

El comodín no tiene por qué ser una sola letra. Un programa puede necesitar manejar dos o más tipos de datos que no tienen nada que ver entre sí dentro de la misma estructura. En esos casos, añadimos más letras separadas por comas (por convención se usan T, U, E, etc.):

```
// Una función que recibe un dato de Tipo 'T' y otro
// de Tipo 'U' totalmente diferentes
fn mostrar_par<T, U>(identificador: T, valor: U) {
    // ...
}
```

La teoría sobre datos genéricos la estudiaremos más adelante pero por ahora debe quedar claro que cuando veamos letras mayúsculas en la definición de estructuras, enumeraciones o funciones significan un tipo de dato cualquiera, genérico.

1.2.4. ¿Cómo se aplica esto a las enumeraciones Option y Result que veremos luego?

Ahora ya estás preparado para entender las herramientas de Rust. Cuando veas:

- Option<T>: Significa que es una caja que puede contener un dato de tipo T (¡el que tú quieras!: un entero, un texto String, una estructura personalizada, etc).

- `Result<T, E>`: Significa que es una enumeración que, si todo va bien, devuelve un dato exitoso de tipo **T**, y si falla, devuelve un error de tipo **E**.

Los genéricos son simplemente eso: comodines que toman su valor real en el mismo instante en el que tú decides utilizarlos en tu programa.

Estas dos enumeraciones están definidos por Rust y por tanto en nuestros programas podremos utilizarlos directamente sin definirlos. Son enumeraciones genéricas predefinidas.

Ciertamente es un concepto bastante abstracto pero cuando lo veas aplicado en algunos ejemplos lo normalizarás rápidamente.

2. Gestión de Errores Profesional en Rust

Como hemos avanzado en la introducción existen muchas causas que pueden provocar un funcionamiento erróneo de un programa. Ya que eliminar esas causas no está entre las facultades del programador, lo que debe hacer es anticiparse y prever un comportamiento seguro del programa para el máximo número de situaciones.

Clasificamos los distintos errores que pueden ocurrir.

2.1. Errores recuperables vs. Irrecuperables

En la programación real, las cosas fallan: un archivo que intentamos abrir resulta que no existe, la red se cae cuando nuestro programa intenta acceder a ella o un usuario introduce letras en un campo de entrada en lugar de números... Rust clasifica los errores en dos categorías:

- **Errores irrecuperables** (panic!): Problemas graves de los que el programa no puede recuperarse (ej. acceder a un índice fuera de los límites de un vector). El programa se detiene inmediatamente.
- **Errores recuperables**: Situaciones que podemos anticipar y solucionar (ej. si intentamos abrir un archivo y resulta que no existe, podemos pedirle al usuario que introduzca otra ruta o incluso, crear el archivo si eso elimina el error).

2.2. El tipo Option<T>

Rust no tiene el valor **null** presente en otros lenguajes, evitando así los famosos errores de «puntero nulo». Cuando un dato puede estar presente o ausente, lo representamos con la enumeración **Option<T>**, que tiene dos variantes:

- **Some(valor)**: Contiene el dato. Es valor.
- **None**: Indica la ausencia del dato.

Para ir fijando las ideas podemos suponer que ese valor que decimos que puede estar presente o ausente, es normalmente el valor devuelto por una función y que se asigna a una variable.

Supongamos que estamos trabajando con **Option<f64>**. En lenguaje simbólico podríamos reflejar la situación de la siguiente forma:

```
fn mi_funcion(parametros)-> Option<f64> {instrucciones}

let mi_variable = funcion(argumentos);
```

Según el esquema anterior, **mi_variable** será de tipo **Option<f64>**. Esta enumeración tiene como hemos mencionado antes dos variantes y **mi_funcion** asignará una de las dos a **mi_variable**. Si le asigna **Some**, dentro del **Some** está el valor **f64** que buscamos y si es **None**, sencillamente el valor no existe. En este caso, **f64** es el tipo de dato que se ajusta a nuestras necesidades pero la enumeración **Option<T>** está definida para cualquier tipo de dato usando el genérico **T**. Es el compilador el que genera una función diferente para cada tipo de dato cuando lo utilizamos.

En el siguiente ejemplo —ver código completo más abajo—, la función **buscar_usuario(id: u32)** simula buscar un usuario a partir de su **id**. Hemos simplificado para que solo exista un usuario que se obtiene con el **id = 10**. La función devuelve un **option<String>**. Veamos los dos casos posibles:

- Si le proporcionamos un **id = 10** (ver más abajo listado del programa **buscar_usuario.rs**), la función devuelve **Some(String::from(«Alicia»))**, es decir, devuelve un **String** con el resultado correcto envuelto en un **Some**. Ya aprendimos a desempaquetar el resultado del **Some** en la función **main** pero lo volveremos a hacer.
- Si le proporcionamos cualquier otro **id** devuelve **None** que interpretamos como ausencia de valor. Esta ausencia de valor no genera ningún problema con las sentencias **println!** que muestran el resultado.
- Observa la instrucción **match** del **main**:

```
let usuario_id = 10;
match buscar_usuario(usuario_id) {
    Some(nombre) => println!("Usuario encontrado: {}", nombre),
    None => println!("Error: El usuario no existe en la base de datos."),
}
```

El **match** chequea los dos posibles valores que puede tomar la función en base al **usuario_id** que le pasemos y establece el código a ejecutar en cada rama.

En la línea:

```
Some(nombre) => println!("Usuario encontrado: {}", nombre)
```

La variable **nombre** atrapa el valor oculto en el **Some** y podemos utilizarla luego en el **println!** a la derecha.

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **buscar_usuario.rs**

```
// Ejemplo sencillo de Option
fn buscar_usuario(id: u32) -> Option<String> {
    if id == 10 {
        Some(String::from("Alicia"))
    } else {
        None
    }
}

fn main() {
    let usuario_id = 10;

    match buscar_usuario(usuario_id) {
        Some(nombre) => println!("Usuario encontrado: {}", nombre),
        None => println!("Error: El usuario no existe en la base de datos."),
    }
}
```



```
}
}
```

Cambia el valor de la variable **let usuario_id = 10;** y observa los mensajes de salida.

2.3. El tipo **Result<T, E>**

Result es otra enumeración definida en Rust para tratar con operaciones que pueden fallar por factores externos y en las que podemos capturar el error.

Sus variantes son:

- **Ok(T)**: La operación fue un éxito y devuelve el resultado de tipo **T** en la variante **Ok**.
- **Err(E)**: La operación falló y devuelve el error de tipo **E** en la variante **Err**.

Supongamos una función que pide al usuario que introduzca el numerador y denominador para calcular un cociente y devuelva el resultado. El problema es, ¿qué pasa si el usuario introduce un cero para el denominador?

Si la función devuelve exactamente el cociente se producirá un error en el programa y se abortará su ejecución porque no se puede dividir por cero.

La solución de Rust a este problema es, por ejemplo, que la función devuelva un tipo **Result<f64, String>**. Dentro de la función, cuando el denominador no es cero se devuelve la variante **Ok(resultado)**, resultado será de tipo **f64**, y si es cero devolvemos por ejemplo **Err(String::from("No se puede dividir por cero"))**

Un caso muy parecido salvo que los valores del numerador y denominador los elegimos nosotros en el código es el del siguiente listado:

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **result_division.rs**

```
// Ejemplo de división segura
fn dividir(dividendo: f64, divisor: f64) -> Result<f64, String> {
    if divisor == 0.0 {
        Err(String::from("No se puede dividir por cero."))
    } else {
        Ok(dividendo / divisor)
    }
}

fn main() {
    match dividir(10.0, 2.0) {
        Ok(resultado) => println!("Resultado: {}", resultado),
        Err(error) => println!("Ocurrió un error: {}", error),
    }
}
```

De nuevo utilizamos la instrucción **match** para desempaquetar el **Result** que devuelve la función **dividir**.

- La variable **resultado** en **Ok(resultado)** recibe el valor **dividendo / divisor** que se le pasa al **Ok** en la función.

- La variable **error** en `Err(error)` recibe el **String** que se le pasa a `Err` en la función.

Como el `Result` tiene dos variantes, `Ok` y `Err`, tenemos que tener las dos en cuenta en el `match` ya que de otra forma el programa no podría ser compilado.

2.4. El tipo de dato unidad

No hemos hablado hasta el momento en los cuadernos del tipo de dato **unidad** porque esperábamos a tener un contexto adecuado para hacerlo.

En Rust, todas las funciones devuelven algo. Bien con una sentencia `return` o bien mediante una expresión en su última línea que no termina con punto y coma. En ambos casos el tipo devuelto se indica en la cabecera de la función.

Si la función no retorna nada de forma explícita, entonces retorna un tipo de dato denominado unidad. El símbolo del tipo de dato unidad es `()` y solo existe un valor de este tipo de dato que se escribe de la misma forma y se denomina también unidad `()`.

La sentencia `Result` que hemos visto antes, **`Result<T, E>`**, algunas veces se utiliza de la siguiente forma **`Result<(), string>`**. Entonces, las dos variantes se escriben:

```
Ok::<>()
Err("String::from"(Cualquier error que pueda ocurrir.))
```

En estos casos, cuando se da la variante `OK` no estamos interesados en recoger ningún resultado. Simplemente notificamos que la operación, sea la que sea, ha ido bien.

Veremos un ejemplo con este planteamiento un poco más adelante.

2.5. El operador de propagación ?

Para evitar encadenar demasiados bloques `match`, Rust ofrece el operador `?`. Lo podemos ver aplicado en la siguiente instrucción perteneciente al listado **interrogacion.rs** más abajo:

```
let resultado = dividir(10.0, 10.0)?;
```

Si el resultado de la función **`dividir`** es `Ok`, el operador `?` extrae el valor y lo asigna a la variable `resultado`; si es `Err`, detiene la función actual, en este caso `dividir`, y devuelve («propaga») el error automáticamente a la función que la llamó. En el listado más abajo podemos ver que la función que llama a `dividir` es `main`.

Para que esto funcione, la función que llama a `dividir`, `main`, debe devolver un `Result` con una variante `Err` del mismo tipo que la función `dividir`, en este caso `String`. De esta forma, cuando se produce un `Err` en `dividir`, esta lo lanza hacia arriba, hacia `main` y esta que admite el mismo tipo de `Err` lo imprime.

La variante `OK()` del `Result` que devuelve `main` es de tipo unidad y, su valor es la unidad. Es consecuencia de que no deseamos hacer nada en `main` cuando todo haya ido bien ya que imprimimos el resultado en la línea anterior.

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **interrogacion.rs**

```
// Ejemplo de división segura
fn dividir(dividendo: f64, divisor: f64) -> Result<f64, String> {
    if divisor == 0.0 {
        Err(String::from("No se puede dividir por cero."))
    } else {
        Ok(dividendo / divisor)
    }
}

fn main() -> Result<(), String>{
    let resultado = dividir(10.0, 0.0)?;
    println!("El resultado es: {resultado}");
    Ok(())
}
```

Salida del programa:

```
Error: "No se puede dividir por cero."
```

La primera parte **Error:** la imprime Rust y el mensaje "**No se puede dividir por cero.**" es el que mandamos desde la función dividir al utilizar `?`.

3. Interacción con el Sistema Operativo y Archivos

Lo que veremos en este tema es solo una pequeña introducción al tratamiento de archivos mediante Rust y a como puede interaccionar con el sistema operativo.

3.1. Leyendo archivos de texto

Nos limitaremos a la lectura de ficheros de texto.

Dado que no podemos trabajar con ficheros en la Playground, crearemos un proyecto de Rust con Cargo. Abre una terminal integrada de Visual Studio Code en la carpeta **proyectos-rust** anteriormente creada o en cualquier otra carpeta.

Ejecuta el siguiente comando para crear un proyecto:

```
cargo new sist_op_archivos
```

Despliega el directorio y crea el fichero **config.txt** al mismo nivel que el fichero **Cargo.toml**.

El fichero **config.txt** es el que vamos a leer desde nuestro programa y puedes poner, por ejemplo, las siguientes líneas:

config.txt

```
sdtv_mode=18
# 0 para NTSC
# 1 para la versión japonesa de NTSC
# 2 para PAL
# 3 para la versión brasileña de PAL
# 16 para NTSC progresivo
# 18 para PAL progresivo
```

 Copia el siguiente código y pégalo en el fichero **main.rs** del proyecto creado borrando previamente todo lo que hubiera en ese archivo.

Fichero: **main.rs**

```
use std::fs;

fn main() {
    let contenido = fs::read_to_string("config.txt");

    match contenido {
        Ok(datos) => println!("Contenido del archivo:\n{}", datos),
        Err(e) => println!("Error al leer el archivo: {}", e),
    }
}
```

Abre una terminal integrada de VS Code en la carpeta del proyecto y ejecuta el programa con la orden **cargo run**. Deberá mostrarte en la terminal el contenido del archivo **config.txt**.

Explicación:

La instrucción **use std::fs;** es necesaria para poder acceder a la función **read_to_string()** que lee el fichero **config.txt**.

Como la operación de lectura puede fallar si escribimos mal el nombre del fichero o si no existe, **read_to_string()** devuelve un **Result** (la función está programada así en Rust) que es asignado a la variable **contenido**.

A continuación, el **match** desempaqueta **contenido** teniendo en cuenta las dos variantes del **Result** que puede contener.

- Si contenido es **OK**, la variable **datos** contiene el fichero leído y se imprime con el **println!**. Podríamos poner a **datos** cualquier otro nombre siempre que utilicemos el mismo a la derecha.
- Si contenido es **Err**, la variable **e** contiene el error y se imprime con **println!**. Podríamos poner a **e** cualquier otro nombre siempre que utilicemos el mismo a la derecha.

Tengamos en cuenta que en este ejemplo, nosotros no hemos escrito la función **read_to_string()** que devuelve el **Result**, es una función de Rust y nosotros solo la utilizamos. En el tema anterior si que escribimos la función **dividir**:

```
fn dividir(dividendo: f64, divisor: f64) -> Result<f64, String> {...}
```

y desempaquetamos de forma análoga el **Result** con **match**.

3.2. Creación y escritura de datos en archivos

Para escribir datos, podemos crear un archivo nuevo o sobrescribir uno existente mediante el uso de **std::fs::File** y **std::io::Write**.

std::fs::File lo necesitamos para utilizar la función **File::create()** y **std::io::Write** para utilizar **write_all()** en **archivo.write_all()**.

- Crea el proyecto de Rust **escribir_en_archivo** en la carpeta **proyectos-rust** igual que hemos hecho en el apartado anterior.

 Copia el siguiente código y pégalo en el fichero **main.rs** del proyecto creado borrando previamente todo lo que hubiera escrito en ese fichero.

Fichero: **main.rs**

```
use std::fs::File;
use std::io::Write;

fn main() {
    let mensaje = "LOG: El sistema se inició correctamente.";

    // 1. Intentamos crear/abrir el archivo
    match File::create("registro.log") {
        Ok(mut archivo) => {
            // 2. Si el archivo se creó con éxito, intentamos escribir los
            bytes
```

```

match archivo.write_all(mensaje.as_bytes()) {
    Ok(_) => {
        // Si todo salió bien
        println!("Datos guardados exitosamente.");
    }
    Err(e) => {
        // Si falló la escritura
        println!("No se pudo escribir en el disco: {}", e);
    }
}
}
Err(e) => {
    // Si falló la creación del archivo
    println!("No se pudo crear el archivo 'registro.log': {}", e);
}
}
}

```

Abre una terminal integrada de VS Code en la carpeta del proyecto y ejecuta el programa con la orden **cargo run**. Deberá crear el archivo **registro.log** al mismo nivel del archivo Cargo.toml.

Explicación

El contenido de la variable **mensaje** es lo que guardaremos en el fichero que crearemos.

La orden **File::create("registro.log")** tiene dos posibles resultados proporcionados por la función de Rust **create()**: **Ok** o **Err** ya que la función puede tener éxito o fallar. Esto forma parte del ecosistema de Rust y se obtiene de la documentación ya que nosotros no hemos programado esta función.

El **match File::create("registro.log")** contempla las dos opciones:

- Si el resultado es **Ok**, en el match desempaquetamos el **Ok(mut archivo)** y obtenemos el identificador mutable **archivo** con el que luego podremos crear y escribir en el fichero registro.log.
- Si el resultado es **Err** se desempaqueta también el **Err(e)** obteniendo el valor de **e** y se imprime el error de fallo en la creación del archivo.

Si el primer match se resolvió con OK intentamos escribir en el archivo para lo cual utilizamos un segundo match:

- **match archivo.write_all(mensaje.as_bytes())**

Aquí de nuevo tenemos las dos posibilidades. Si el **match** se resuelve en **Ok(_)**, con el guión bajo indicamos que no estamos interesados en desempaquetar ningún dato que pueda traer el Ok. Simplemente imprimimos el mensaje de que todo ha salido bien. Si se resuelve en **Err(e)** aquí sí que desempaquetamos **Err**, obtenemos el valor de **e** y lo utilizamos en el mensaje de consola.

El programa anterior puede simplificarse utilizando una función que acepta el mensaje a guardar en el fichero y el operador ?. Puedes consultarlo en:

Proyecto: escribir_en_archivo_2

El listado es el siguiente:

```

use std::fs::File;
use std::io::Write;

fn guardar_registro(mensaje: &str) -> std::io::Result<()> {
    let mut archivo = File::create("registro.log"?;
    archivo.write_all(mensaje.as_bytes())?;
    Ok(())
}

fn main() {
    if let Err(e) = guardar_registro("LOG: El sistema se inició
correctamente.") {
        println!("No se pudo escribir en el disco: {}", e);
    } else {
        println!("Datos guardados exitosamente.");
    }
}

```

Explicación

Al ejecutar la primera orden del main:

```
if let Err(e) = guardar_registro("LOG: El sistema se inició correctamente.")
```

Si las dos líneas a las que aplicamos el operador ? en la función guardar_registro():

```

let mut archivo = File::create("registro.log"?;
archivo.write_all(mensaje.as_bytes())?;

```

se ejecutan sin problemas, se crea el fichero con el contenido especificado y se ejecuta el else del main porque no se generó Err.

Si alguna de las dos funciones **create()** o **write_all()** se resuelve en **Err**, el error se propaga a quien la llamó, que es el main. En el main desempaquetamos con **if let Err(e)** el error imprimimos un mensaje por la consola.

4. Gestión con Cargo y el Ecosistema de Rust

Cargo es el gestor de paquetes, compilador y sistema de construcción de Rust. La construcción de un programa es algo similar a la unión de varias piezas (programas) para formar un único programa principal.

Entonces podemos definir a Cargo como la herramienta «todo en uno» que acompaña al lenguaje para facilitar la vida del desarrollador, encargándose de automatizar prácticamente todas las tareas relacionadas con la gestión de un proyecto.

4.1. ¿Cuáles son las funciones principales de Cargo?

Cargo centraliza todo el flujo de trabajo de un programador de Rust a través de cuatro pilares básicos:

- **Gestión de dependencias:** Descarga, actualiza y compila automáticamente las librerías externas (llamadas crates) que tu proyecto necesita desde el registro oficial **crates.io**.
- **Sistema de construcción:** Compila tu código fuente con un solo comando, configurando de forma transparente los parámetros optimizados para desarrollo o para producción.
- **Orquestador de pruebas:** Busca y ejecuta de forma nativa todas las pruebas unitarias y de integración que hayas escrito para verificar que tu código funcione bien.

Nosotros no escribiremos pruebas unitarias.

- **Generador de proyectos:** Crea la estructura de carpetas inicial estándar y genera los archivos de configuración necesarios para empezar un programa desde cero.

4.2. Los comandos esenciales de Cargo

En el día a día, casi nunca interactúas con el compilador puro (rustc), sino que utilizas Cargo a través de la terminal mediante estos comandos:

- **cargo new mi_proyecto:** Crea una carpeta con un proyecto nuevo listo para programar.
- **cargo build:** Compila el proyecto y genera el archivo ejecutable.
- **cargo run:** Compila el código (si ha habido cambios) y lo ejecuta inmediatamente.
- **cargo check:** Revisa rápidamente el código en busca de errores de compilación sin perder tiempo en generar el archivo ejecutable (ideal mientras estás programando).
- **cargo test:** Ejecuta todos los tests de tu aplicación.

Cuando creamos un proyecto con la orden **cargo new mi_proyecto** se genera un directorio para el proyecto con dos ficheros que vamos a comentar a continuación.

4.3. Cargo.toml (El manifiesto del desarrollador)

El archivo Cargo.toml (escrito con la primera letra en mayúscula y extensión .toml) es un fichero generado automáticamente por Cargo. No está pensado para ser manipulado manualmente aunque se puede hacer.

En este archivo tú especificas:

- **Metadatos:** El nombre de tu programa, la versión actual, el autor y la edición de Rust que usas.
- **Tus intenciones:** Qué librerías externas (crates) quieres usar y qué versiones estás dispuesto a aceptar.

Ejemplo típico de un Cargo.toml:

```
[package]
name = "mi_proyecto"
version = "0.1.0"
edition = "2024"

[dependencies]
# Aquí se añaden las librerías del ecosistema
csv = "1.3"
serde = "1.0"
```

4.4. Cargo.lock (El registro exacto del sistema)

El archivo **Cargo.lock** (completamente autogenerado por Cargo) es el que garantiza que tu código se compile exactamente igual en cualquier ordenador del mundo.

Tú no debes editar este archivo a mano. Cargo lo gestiona solo.

Guarda las versiones exactas (por ejemplo, 1.0.199 en lugar de solo 1.0).

Si otra persona descarga tu proyecto un año después, Cargo.lock asegura que use exactamente las mismas versiones que usaste tú, evitando que una actualización rota en una librería externa rompa tu programa de repente.

4.5. ¿Qué son las crates y Crates.io?

En Rust, los paquetes de código o librerías se llaman crates. La comunidad publica miles de soluciones de código abierto en la web oficial **crates.io**. En lugar de reinventar la rueda (por ejemplo, programar nuestro propio lector de ficheros CSV «tipo hoja de cálculo» o generador de gráficos), añadimos la línea correspondiente en [dependencies] y Cargo se encarga de descargar el crate indicado, verificarlo y compilarlo de forma totalmente transparente para que lo podamos usar.

5. Procesamiento de Datos y Proyectos Prácticos

Hasta este punto del camino, hemos explorado los cimientos teóricos que hacen de Rust un lenguaje único: la rigurosidad de su sistema de tipos, las garantías inquebrantables de su modelo de propiedad (ownership) y la elegancia de su gestión de errores. Sin embargo, la verdadera potencia de estas herramientas no se aprecia en el aislamiento de los ejemplos de laboratorio, sino en el terreno de las aplicaciones del mundo real.

Este cuarto capítulo marca la transición definitiva de la teoría a la práctica. El ecosistema de Rust no solo destaca por su seguridad en memoria, sino también por su capacidad para procesar grandes volúmenes de información con un rendimiento predecible y una eficiencia implacable. A lo largo de las siguientes páginas, descubriremos cómo conectar las estructuras de datos nativas con el almacenamiento persistente, cómo leer y transformar flujos de información externos y cómo estructurar código modular que sea fácil de mantener a medida que crece.

Para consolidar estos conceptos, dejaremos de lado los fragmentos de código inconexos y nos enfocaremos en la construcción de proyectos prácticos orientados a resolver problemas reales. Utilizaremos Cargo no solo como un compilador, sino como un aliado estratégico para integrar dependencias del ecosistema y organizar nuestra arquitectura. Al finalizar este capítulo, habrás transformado las restricciones del compilador en tus mayores aliadas, adquiriendo la confianza necesaria para diseñar, estructurar y desplegar tus propias aplicaciones e infraestructuras en Rust.

5.1. Parseo numérico

Para convertir **cadenas de texto** (String o &str) obtenidas del usuario (o de archivos) en números reales manipulables matemáticamente, se utiliza el método `.parse()`. Como esta operación **puede fallar** (por ejemplo si el texto contiene letras), **siempre devuelve un Result**.

 Copia el siguiente código y ejecútalo en la Playground

Fichero: **parse.rs**

```
fn main() {
    let texto_numero = String::from("42");

    // El compilador infiere que queremos un i32 gracias
    // a la anotación de tipo
    let numero: Result<i32, _> = texto_numero.parse();

    match numero {
        Ok(n) => println!("El número {} multiplicado por 2 es: {}", n, n * 2),
        Err(_) => println!("No se pudo convertir la cadena a número."),
    }
}
```

Obtendrás el siguiente resultado:

«El número 42 multiplicado por 2 es: 84»

Lo que demuestra que la función **.parse()** ha convertido el String **texto_numero** en un número **i32**, ya que lo hemos podido multiplicar por 2 que es por defecto **i32** (no hace falta escribir **2i32**).

Analicemos la línea:

```
let numero: Result<i32, _> = texto_numero.parse();
```

Con **Result<i32, _>** le estás diciendo al compilador: sabemos que la función **.parse()** va a devolver un **Result**. Yo elijo el tipo del **Ok** (**i32**) y el compilador que averigüe el tipo del **Err** investigando el contexto de la operación (equivalente a mirar los manuales).

Se procede de esta forma porque si no, tendríamos que especificar nosotros el tipo del error y en este caso tendríamos que haber escrito:

```
// Código completo sin el guion bajo (más largo de escribir):
let numero: Result<i32, std::num::ParseIntError> = texto_numero.parse();
```

Tendríamos que saber, o si no averiguar, que el tipo de **Err** para nuestro caso es: **std::num::ParseIntError**. Tendríamos que mirar nosotros los manuales.

En el **match** se desempaqueta el **Ok** extrayendo **n** que es el **número** en **i32** convertido a partir del String **texto_numero**. La parte de **Err** no nos interesa y le pasamos un guión bajo.

5.2. Proyecto 1: El Gestor de Tareas Pendientes (To-Do-List). Grabar en disco

Este programa interactivo por consola te permitirá añadir tareas y guardarlas de forma persistente para que no se pierdan al cerrar la aplicación.

- Abre una terminal integrada de VS Code en la carpeta **proyectos-rust** (o en cualquier otra) y crea el proyecto de Rust **lista_tareas** con la orden:

```
cargo new lista_tareas
```

 Copia el siguiente código y pégalo en el fichero **main.rs** del proyecto creado, borrando previamente todo lo que hubiera escrito en ese fichero.

Fichero: **main.rs**

```
use std::fs::{OpenOptions, read_to_string};
use std::io::{self, Write};

const ARCHIVO_TAREAS: &str = "tareas.txt";

fn mostrar_tareas() {
    println!("\n--- MIS TAREAS PENDIENTES ---");
    match read_to_string(ARCHIVO_TAREAS) {
        Ok(contenido) => {
            if contenido.trim().is_empty() {
                println!("No tienes tareas pendientes.");
            } else {
```

```

        println!("{}", contenido);
    }
}
Err(_) => println!("No se encontró archivo previo. ¡Añade tu primera
tarea!"),
}
}

fn agregar_tarea(tarea: &str) -> io::Result<()> {
    // Abrimos el archivo en modo "añadir" (append) o lo creamos si no existe
    let mut archivo = OpenOptions::new()
        .write(true)
        .append(true)
        .create(true)
        .open(ARCHIVO_TAREAS)?;

    writeln!(archivo, "- {}", tarea)?;
    Ok(())
}

fn main() {
    loop {
        mostrar_tareas();
        println!("\nOpciones: [1] Añadir tarea [2] Salir");
        print!("Selecciona una opción: ");
        io::stdout().flush().unwrap();

        let mut opcion = String::new();
        io::stdin().read_line(&mut opcion).unwrap();

        match opcion.trim() {
            "1" => {
                print!("Escribe la nueva tarea: ");
                io::stdout().flush().unwrap();
                let mut nueva_tarea = String::new();
                io::stdin().read_line(&mut nueva_tarea).unwrap();

                if let Err(e) = agregar_tarea(nueva_tarea.trim()) {
                    println!("Error al guardar la tarea: {}", e);
                }
            }
            "2" => {
                println!("¡Hasta luego!");
                break;
            }
            _ => println!("Opción no válida."),
        }
    }
}

```

📌 Nota: Empezamos ahora un proceso destinado a que entiendas la mayor parte posible del programa. En la carpeta 'src' del proyecto, junto al archivo main.rs hay otro archivo con el nombre 'listado_comentado.rs' que contiene el mismo programa anterior pero

comentado prácticamente línea por línea. Puedes apoyarte también en este archivo para la comprensión del programa.

Ejecuta el programa

Antes de empezar a investigar cómo funciona el programa, vamos a ver qué es lo que hace. Abre una terminal integrada en la carpeta del proyecto (**lista_tareas**) y ejecuta la orden **cargo run**. Debe aparecer en pantalla el siguiente mensaje:

```
--- MIS TAREAS PENDIENTES ---
No se encontró archivo previo. ¡Añade tu primera tarea!

Opciones: [1] Añadir tarea [2] Salir
Selecciona una opción:
```

- Pulsa 1 y a continuación Enter para seleccionar la opción [1]

El programa contesta con:

```
--- MIS TAREAS PENDIENTES ---
No se encontró archivo previo. ¡Añade tu primera tarea!

Opciones: [1] Añadir tarea [2] Salir
Selecciona una opción: 1
Escribe la nueva tarea:
```

Primero lista las tareas que tienes grabadas en un fichero (Todavía no tienes tareas ni fichero) y te pide **Escribe la nueva tarea**:

Puedes escribir cualquier cosa, como por ejemplo «Estudiar el Cuaderno 1 de Rust» y pulsas Enter. En ese momento se creará el fichero **tareas.txt** en el mismo nivel que Cargo.toml y se guardará la tarea en el fichero.

Podrás seguir añadiendo tareas a través del menú o salir del programa. Cada vez que añadas una tarea el programa te listará todas las tareas que tienes guardadas y te presentará el menú de añadir tarea o salir.

Bien, una vez sabemos qué hace el programa vamos a ver cómo lo hace. Algunas cosas no las podremos explicar en detalle porque se apoyan en conceptos que no hemos explicado todavía. No obstante, en el archivo **listado_comentado.rs** están comentadas prácticamente todas las líneas.

Empezamos

1) Las sentencias use:

```
use std::fs::{OpenOptions, read_to_string};
use std::io::{self, Write};
```

Son necesarias para poder utilizar nombres de funciones del ecosistema Rust que aparecen en el listado.

2) Definimos la constante:

```
const ARCHIVO_TAREAS: &str = "tareas.txt";
```

Las constantes se definen con la palabra **const** y un nombre en mayúsculas. Son similares a las variables pero su valor no puede cambiar durante la ejecución del programa. Son visibles desde cualquier punto del programa, por eso se llaman globales y existen desde el instante en que se ejecuta el programa hasta que termina.

Son prácticas porque si utilizamos una constante en muchos sitios en el interior del programa, podemos cambiar ese valor en todos los sitios a la vez modificando la línea de arriba de su definición y cambiando allí el valor.

3) La función `mostrar_tareas()`{...

Su funcionamiento está basado en la línea:

```
match read_to_string(ARCHIVO_TAREAS) {...
```

El `match` desempaqueta las dos variantes de `Result` que puede devolver `read_to_string(ARCHIVO_TAREAS)`:

1. Con `Ok(contenido)` => instrucciones...
 - `read_to_string` ha tenido éxito leyendo `ARCHIVO_TAREAS` y la variable `contenido` contiene precisamente el contenido del archivo.
 - `contenido.trim().is_empty` quita los espacios que pueda tener el texto al principio y al final y comprueba si está vacío.
 - finalmente imprime un mensaje entre dos opciones mediante un `if-else`
2. Con `Err(_)` ignora el valor del error e imprime un mensaje personalizado

4) La función `agregar_tarea(tarea: &str) -> io::Result<()> {...`

- Añade una nueva línea de texto al final de nuestro archivo.
- Retorna un `"io::Result<()>"` permitiendo propagar errores de I/O mediante el operador `"?"`.
- Está comentada línea por línea en el archivo `listado_comentado.rs`.

5) El método `main`

- Crea primero que nada un bucle `loop`
- En el interior del bucle, llama al empezar a la función `mostrar_tareas()` y a continuación pide al usuario que elija una opción introduciendo un 1 o un 2. La elección del usuario se coloca en la variable **`opcion`**.
- Con `match opcion.trim()` desempaqueta el valor de `opcion` después de quitarle los posibles espacios que tenga al principio y al final.
- Si el valor de `opcion` es **`1`**, pide al usuario que escriba el nombre de la nueva tarea y lo coloca en la variable `nueva_tarea`.

- Con la instrucción `if let Err(e)` a verigua si en **agregar_tarea(nueva_tarea.trim())** se ha producido un error. Si es así lo imprime y sale. Si no hubo error, la nueva_tarea se ha agredado al fichero **tareas.txt**.
- Si el valor de la opción es **2** sale del programa.
- Con el guión bajo `_` se recoge cualquier otro valor de opcion y se presenta el mensaje de opcion no válida.
- Ver **listado_comentado.rs** para comentarios más detallados.

5.3. Proyecto 2: Herramienta CLI para automatización del sistema

Este programa interactúa directamente con los argumentos proporcionados por la terminal del sistema operativo utilizando **std::env::args**. Sirve para simular una herramienta administrativa que automatiza tareas cotidianas como por ejemplo crear o borrar un fichero.

Creación del proyecto

- Abre una terminal integrada de VS Code en la carpeta **proyectos-rust** (o en cualquier otra) y crea el proyecto de Rust **herramientas_cli** con la orden:

```
cargo new herramientas_cli
```

 Copia el siguiente código y pégalo en el fichero **main.rs** del proyecto creado, borrando previamente todo lo que hubiera escrito en ese fichero.

Fichero: **main.rs**

```
use std::env;
use std::fs;

fn main() {
    // Captura los argumentos de la línea de comandos
    // (ej: cargo run -- limpiar archivo.txt)
    let argumentos: Vec<String> = env::args().collect();

    // Imprimimos el vector argumentos para ver lo que contiene
    println!("{:?}", argumentos);

    if argumentos.len() < 3 {
        println!("Uso incorrecto del programa.");
        println!("Ejemplo: cargo run -- <accion> <nombre_archivo>");
        println!("Acciones disponibles: eliminar, crear");
        return;
    }

    let accion = &argumentos[1];
    let nombre_archivo = &argumentos[2];

    match accion.as_str() {
        "crear" => {
            match fs::File::create(nombre_archivo) {
                Ok(_) => println!("Automatización: Archivo '{}' creado con éxito.", nombre_archivo),
                Err(e) => println!("Error al crear el archivo: {}", e),
            }
        }
    }
}
```

```

    }
    "eliminar" => {
        match fs::remove_file(nombre_archivo) {
            Ok(_) => println!("Automatización: Archivo '{}' eliminado de
forma segura.", nombre_archivo),
            Err(e) => println!("Error al eliminar el archivo: {}", e),
        }
    }
    _ => println!("Acción desconocida. Prueba con 'crear' o 'eliminar.'),
}
}

```

❗ **Nota:** Empezamos ahora un proceso destinado a que entiendas la mayor parte posible del programa. En la carpeta 'src' del proyecto, junto al archivo main.rs hay otro archivo con el nombre 'listado_comentado.rs' que contiene el mismo programa anterior pero comentado prácticamente línea por línea. Puedes apoyarte también en este archivo para la comprensión del programa.

Ejecuta el programa

Al igual que en el proyecto anterior, antes de empezar a investigar cómo funciona el programa, vamos a ver qué es lo que hace.

Abre una terminal integrada en la carpeta del proyecto (**herramientas_cli**) y en lugar de ejecutar la orden `cargo run`. como hacemos siempre, ejecuta la siguiente orden:

cargo run -- crear ejemplo.txt

Al pulsar return el programa crea el fichero ejemplo.txt al mismo nivel del fichero Cargo.toml.

Ahora podemos ejecutar la orden:

cargo run -- eliminar ejemplo.txt

Y borraremos el fichero creado anteriormente. Por tanto, a través de comandos de texto entregados a nuestro programa, estamos ejecutando órdenes sobre nuestro directorio de ficheros como podríamos hacer directamente desde el sistema operativo.

Explicación

1) La línea:

```
let argumentos: Vec<String> = env::args().collect();
```

.collect() es necesario para construir el vector a partir de `env::args()` que es un iterador.

Captura las distintos parámetros entregados a la orden `cargo run`. La primera orden que hemos ejecutado ha sido: **cargo run -- crear ejemplo.txt** y al imprimir el vector **argumentos** obtenemos:

`["target/debug/herramientas_cli", "crear", "ejemplo.txt"]` (compruébalo en tu terminal)

Por tanto, las variables que nos interesan son:

- `let accion = &argumentos[1];`
- `let nombre_archivo = &argumentos[2];`

2) A continuación tenemos un `match` externo que crea dos ramas de ejecución de flujo según el valor de la variable `accion`. Una para el valor **crear** y otra para **eliminar**.

- En la opción **crear**, otro `match` interno desempaqueta el `Result` que entrega la función **create** e imprime sendos mensajes según el resultado.
- En la opción **eliminar** se procede análogamente con el resultado de la función **remove_file**.

5.4. Atrapar varios tipos de errores en un único Result

Vamos a explicar por qué a veces una función devuelve un `Result` de la forma:

```
Result<f64, Box<dyn Error>>
```

Es un concepto avanzado y si se quiere, se puede admitir que el motivo es porque la función puede devolver errores de distintos tipos y con el `Result<T, U>` que hemos utilizado hasta ahora, el error puede ser de un tipo **U** cualquiera pero no de distintos tipos a la vez.

Si quieres seguir con la explicación, lo primero en observar es que el principal beneficio que obtenemos es la **composición de errores con el operador ?**.

Si tienes una función que lee un archivo (puede producir el error `std::io::Error`) y si luego parsea un número (puede producir el error `std::num::ParseIntError`), no puedes devolver ambos tipos directamente en el `Result<T, U>`. Con **U** solo podemos representar un tipo de error pero no dos. Al usar `Box<dyn std::error::Error>`, Rust convierte automáticamente ambos errores al mismo tipo común.

Ejemplo de código

```
use std::fs::File;
use std::io::Read;
use std::error::Error;

// Usamos Box<dyn Error> para poder devolver cualquier tipo de error
fn leer_numero_de_archivo() -> Result<i32, Box<dyn Error>> {
    let mut archivo = File::open("numero.txt"?); // Puede lanzar std::io::Error
    let mut contenido = String::new();

    archivo.read_to_string(&mut contenido)?; // Puede lanzar std::io::Error

    let numero: i32 = contenido.trim().parse()?; // Puede lanzar ParseIntError

    Ok(numero)
}
```

5.5. Proyecto 3: Analizador Estadístico de archivos .csv

Este último proyecto te introducirá en el uso del ecosistema real de Rust. Utilizaremos la crate externa **csv** (que previamente añadirás al fichero **Cargo.toml** del proyecto). El programa procesará el archivo de datos numéricos **datos.csv** estructurado en una columna e imprimirá el promedio estadístico de los mismos.

El fichero de datos es el que se muestra a continuación:

datos.csv

```
calificacion
15
18
12
20
```

Creación del proyecto

- Abre una terminal integrada de VS Code en la carpeta **proyectos-rust** (o en cualquier otra) y crea el proyecto de Rust **datos_csv** con la orden:

```
cargo new datos_csv
```

- Edita el fichero Cargo.toml de forma que la sección [dependencies] quede como se muestra abajo:

```
[package]
name = "datos_csv"
version = "0.1.0"
edition = "2024"

[dependencies]
csv = "1.3"
```

- Crea un fichero **datos.csv** con el contenido que se muestra al principio y colócalo en el mismo nivel que Cargo.toml.

 Copia el siguiente código y pégalo en el fichero **main.rs** del proyecto creado, borrando previamente todo lo que hubiera escrito en ese fichero.

Fichero: **main.rs**

```
// Nota: Requiere la dependencia 'csv = "1.3"' en Cargo.toml
use std::error::Error;
use std::fs::File;

fn calcular_promedio_csv() -> Result<f64, Box<dyn Error>> {
    // Abrimos el archivo de datos
    let archivo = File::open("datos.csv")?;

    // Inicializamos el lector de la librería externa 'csv'
    let mut lector = csv::Reader::from_reader(archivo);

    let mut suma = 0.0;
    let mut total_elementos = 0;

    // Iteramos por cada registro/fila del archivo de manera segura
    for resultado in lector.records() {
```

```

        let registro = resultado?;
        // Tomamos el primer valor de la fila (columna 0)
        if let Some(valor_texto) = registro.get(0) {
            let valor: f64 = valor_texto.trim().parse()?;
            suma += valor;
            total_elementos += 1;
        }
    }

    if total_elementos == 0 {
        return Ok(0.0);
    }

    Ok(suma / total_elementos as f64)
}

fn main() {
    match calcular_promedio_csv() {
        Ok(promedio) => println!("--- ANALIZADOR ESTADÍSTICO ---\nEl promedio
del archivo es: {:.2}", promedio),
        Err(e) => println!("Error procesando el archivo CSV: {}", e),
    }
}

```

Ejecuta el programa

Abre una terminal integrada en la carpeta de proyecto y ejecuta lo orden cargo run. Deberás obtener lo siguiente en la terminal:

```

--- ANALIZADOR ESTADÍSTICO ---
El promedio del archivo es: 16.25

```

Explicación

La función;

- **calcular_promedio_csv() -> Result<f64, Box<dyn Error>>**

- Ya hemos explicado en el apartado anterior la utilización de Box<dyn Error> en el Result.

Las líneas:

- let archivo = File::open("datos.csv");
- let mut lector = csv::Reader::from_reader(archivo);

Construyen la variable **lector** que representa el fichero **datos.csv**.

Al entrar en el bucle for hay que tener en cuenta que por defecto, la función **csv::Reader::from_reader(archivo)** asume que la primera fila de cualquier archivo CSV es siempre una cabecera (texto con los nombres de las columnas). Por tanto la instrucción:

for resultado in lector.records()

lector.records(), que representa las filas del fichero datos.csv, empieza por el primer dato numérico (15) saltándose el texto **calificacion**.

La instrucción:

```
if let Some(valor_texto) = registro.get(0) {...
```

Asume que **registro.get(0)** va a proporcionar la variante **Some** de un **Option** y desempaqueta el valor en la variable **valor_texto**. Luego fuerza una conversión de esta variable a **f64**.

La instrucción:

```
let valor: f64 = valor_texto.trim().parse()?;
```

LLeva el operador **?** al final porque si en la línea anterior, por equivocación, **registro.get(0)** fuera un texto en lugar de un número, el operador **?** sería el encargado de lanzar el error hacia el **main**.

6. Ejercicios resueltos

Aquí tienes 5 ejercicios prácticos resueltos, diseñados específicamente para el nivel de este Cuaderno 4 con explicaciones paso a paso para que asimiles la gestión de errores y el manejo de archivos.

6.1. Enunciados

6.1.1. Ejercicio 1: El Validador de Edades (Uso avanzado de `Option<T>`)

- **Enunciado:** Escribe una función llamada `obtener_edad_voto(edad: i32) -> Option<i32>` que reciba la edad de una persona. Si la edad es menor de 18 años o directamente imposible (negativa o mayor a 120), debe retornar `None`. Si es apta para votar (18 o más), debe retornar la edad envuelta en `Some`. En el main, simula el caso con varios valores y muestra un mensaje amigable.

6.1.2. Ejercicio 2: Conversión Segura de Temperatura (Uso de `Result<T, E>`)

- **Enunciado:** En física, el «Cero Absoluto» es la temperatura más baja posible (-273.15 °C). Crea una función `celsius_a_kelvin(celsius: f64) -> Result<f64, string>`. Si la temperatura introducida es inferior al cero absoluto, debe devolver un error explicativo. En caso contrario, debe sumar 273.15 y devolver el valor envuelto en el `Ok`.

6.1.3. Ejercicio 3: Copia de Seguridad de Archivos (Manejo del Sistema de Archivos y del operador `?`)

- **Enunciado:** Desarrolla una función llamada `respaldar_archivo() -> std::io::Result<()>` que intente leer el contenido de un archivo llamado `notas.txt` y escribirlo íntegramente en un archivo nuevo llamado `notas_bak.txt`. Utiliza el operador `?` para propagar los errores de lectura y escritura.

6.1.4. Ejercicio 4: Calculadora Parser Robusta (Manejo de strings y entradas numéricas)

- **Enunciado:** Diseña un programa que reciba una cadena de texto que representa una suma simple (por ejemplo, «15+23»). El programa debe extraer ambos números, convertirlos a enteros (`i32`), sumarlos y mostrar el resultado. Si la cadena no tiene el formato correcto o no contiene números válidos, debe interceptar el error sin colapsar.

6.1.5. Ejercicio 5: Registro de Visitas con Fecha Simulada (Escritura persistente)

Enunciado: Crea un programa que pida al usuario su nombre por consola y lo guarde en un archivo llamado `visitas.txt`. Cada nombre debe guardarse en una línea nueva. El archivo no debe borrarse cada vez que se abre el programa; los nuevos nombres deben añadirse al final del documento (modo `append`).

6.2. Soluciones

6.2.1. Ejercicio 1: El Validador de Edades (Uso avanzado de `Option<T>`)

Código de la Solución

```
fn obtener_edad_voto(edad: i32) -> Option<i32> {
    if edad >= 18 && edad <= 120 {
        Some(edad)
    } else {
        None
    }
}

fn main() {
    let casos_prueba = vec![16, 25, -5, 130];

    for edad in casos_prueba {
        match obtener_edad_voto(edad) {
            Some(e) => println!("Edad {} años: ¡Voto permitido!", e),
            None => println!("Edad {} años: Registro rechazado (Menor de edad o
valor inválido).", edad),
        }
    }
}
```

Explicación Didáctica

- **El tipo Option como filtro:** El ejercicio demuestra cómo Option no solo sirve para la «ausencia» de datos, sino también para descartar activamente datos que no cumplen con las reglas de negocio (valores fuera de rango).
- **El uso de match:** Te obliga a cubrir ambas variantes (Some y None) del enumerado Option, garantizando que el programa nunca intente procesar un voto procedente de una edad inválida como -5.

6.2.2. Ejercicio 2: Conversión Segura de Temperatura (Uso de Result<T, E>)

Código de la Solución

```
fn celsius_a_kelvin(celsius: f64) -> Result<f64, String> {
    if celsius < -273.15 {
        Err(String::from("Error: La temperatura está por debajo del cero
absoluto (-273.15 °C)."))
    } else {
        Ok(celsius + 273.15)
    }
}

fn main() {
    let temperaturas = vec![25.0, -300.0, 0.0];

    for t in temperaturas {
        match celsius_a_kelvin(t) {
            Ok(k) => println!("{:.2} °C equivalen a {:.2} K", t, k),
            Err(mensaje_error) => println!("{}", mensaje_error),
        }
    }
}
```

Explicación Didáctica

- **Diferencia con Option:** Aquí no usamos Option porque si algo falla, queremos explicar por qué ha fallado. La variante Err transporta un String con el motivo exacto del fallo físico.
- **Manejo explícito:** Al procesar el vector con un match, el programa maneja con gracia el valor -300.0 imprimiendo la advertencia en lugar de romperse.

6.2.3. Ejercicio 3: Copia de Seguridad de Archivos (Manejo del Sistema de Archivos y ?)

Código de la Solución

```
use std::fs;

fn respaldar_archivo() -> Result<(), std::io::Error> {
    // 1. Intentamos leer el archivo origen
    let contenido = fs::read_to_string("notas.txt")?;

    // 2. Intentamos escribir en el archivo destino
    fs::write("notas_bak.txt", contenido)?;

    // 3. Si todo va bien, devolvemos la unidad que no es más
    // que una tupla vacía envuelta en Ok
    Ok(())
}

fn main() {
    // Para probar el código, primero creamos un notas.txt simulado
    let _ = fs::write("notas.txt", "Contenido importante de clase.");

    match respaldar_archivo() {
        Ok(_) => println!("¡Copia de seguridad realizada con éxito!"),
        Err(e) => println!("Error crítico durante el respaldo: {}", e),
    }
}
```

Explicación Didáctica

El poder de ?: El operador ? limpia el código. Sin ?, necesitaríamos dos bloques **match** anidados (uno para leer y otro para escribir en el fichero), lo que haría el código muy farragoso. Con el operador ? y el tipo adecuado en el Result **std::io::Error**, cualquiera de los dos errores que se produzcan se propagan al main y se tratan allí con un solo match.

El tipo de retorno (): El «tipo unidad» () dentro de Result<(), Error>, se utiliza para indicar que todo ha ido bien y no porque se necesite devolver un valor matemático.

6.2.4. Ejercicio 4: Calculadora Parser Robusta (Manejo de strings y entradas numéricas)

Código de la Solución

```
fn sumar_cadena(operacion: &str) -> Result<i32, String> {
    // Dividimos la cadena usando el carácter '+' como separador
    let partes: Vec<&str> = operacion.split('+').collect();

    if partes.len() != 2 {
```

```

        return Err(String::from("Formato inválido. Debe ser: número+número"));
    }

    // Intentamos parsear ambos operandos eliminando espacios en blanco
    adicionales
    let num1: i32 = partes[0].trim().parse()
        .map_err(|_| String::from("El primer operando no es un número
    válido."))?;

    let num2: i32 = partes[1].trim().parse()
        .map_err(|_| String::from("El segundo operando no es un número
    válido."))?;

    Ok(num1 + num2)
}

fn main() {
    let pruebas = vec!["15+23", " 8 + 12 ", "10+tres", "45-5"];

    for p in pruebas {
        match sumar_cadena(p) {
            Ok(resultado) => println!("Operación '{}' = {}", p, resultado),
            Err(e) => println!("Error en '{}': {}", p, e),
        }
    }
}

```

Explicación Didáctica

- **Robustez en Parsing:** El uso de `.trim()` enseña a los alumnos a limpiar los datos de entrada de los usuarios (quitando espacios no deseados).
- **El método `.map_err()` (Opcional/Avanzado):** Permite transformar el error técnico que da `.parse()` (un `ParseIntError`) en un mensaje de texto personalizado y entendible por cualquier usuario antes de usar el operador `?`.

Aclaremos esto último.

La función `.parse()` intenta convertir el texto en un número `i32`. Esto devuelve un `Result`:

- Si tiene éxito: Devuelve `Ok(número)`.
- Si falla: Devuelve `Err(ParseIntError)` (por ejemplo, si el texto era "hola" en lugar de "23" o similar).
- `ParseIntError` es lo que hemos llamado antes un error técnico.

Al aplicar `.map_err(...)`

```

let num1: i32 = partes[0].trim().parse()
    .map_err(|_| String::from("El primer operando no es un número válido."))?;

```

El error técnico es sustituido por el `String` que vemos a la derecha y si el operador `?` actúa porque el `Result` dio error, lo que se transmite al `main` es el `String`, no el error técnico.

6.2.5. Ejercicio 5: Registro de Visitas con Fecha Simulada (Escritura persistente) Código de la Solución


```

use std::fs::OpenOptions;
use std::io::{self, Write};

fn registrar_usuario(nombre: &str) -> io::Result<()> {
    // Abrimos el archivo permitiendo la escritura y forzando el añadido al
    final (append)
    let mut archivo = OpenOptions::new()
        .write(true)
        .append(true)
        .create(true) // Si no existe, lo crea automáticamente
        .open("visitas.txt")?;

    // Escribimos el nombre seguido de un salto de línea
    writeln!(archivo, "Usuario: {}", nombre)?;
    Ok(())
}

fn main() {
    println!("Por favor, introduce tu nombre de estudiante:");

    let mut entrada = String::new();
    io::stdin().read_line(&mut entrada).expect("Error al leer de la consola");
    let nombre_limpio = entrada.trim();

    if nombre_limpio.is_empty() {
        println!("No has introducido ningún nombre.");
        return;
    }

    match registrar_usuario(nombre_limpio) {
        Ok(_) => println!("¡Tu visita ha sido registrada en 'visitas.txt'!"),
        Err(e) => println!("No se pudo acceder al registro: {}", e),
    }
}

```

Explicación Didáctica

- **Diferencia entre `File::create` y `OpenOptions`:** Es crucial entender que **`File::create`** «machaca» (sobrescribe) el archivo desde cero. Para logs, agendas o bases de datos orientadas a registros, se necesita usar **`OpenOptions`** con la propiedad **`.append(true)`**.
- Hemos usado el patrón constructor (Builder Pattern) de `OpenOptions` para configurar el acceso al archivo:
 - `.write(true)` -> Habilita permisos de escritura.
 - `.append(true)` -> Posiciona el puntero al final del archivo para no sobrescribir lo existente.
 - `.create(true)` -> Si el archivo “visitas.txt” no existe, el sistema operativo lo creará automáticamente.
- **Interactividad básica:** Conecta los conceptos del sistema de archivos con la entrada estándar (`stdin`) que ya conocen de los cuadernos anteriores.

7. Ejercicios propuestos

7.1. Ejercicio 1: El Control de Altitud (Práctica con `Option<T>`)

- **Objetivo:** Utilizar `Option<T>` para gestionar límites de seguridad físicos en un entorno de simulación aérea.
- **Enunciado:** Desarrolla un programa para un dron de reconocimiento. Crea una función llamada `verificar_altitud(metros: i32) -> Option<i32>`. El dron opera de forma segura únicamente entre los 10 metros y los 500 metros de altura. Si la altitud está en ese rango, devuelve el valor envuelto en `Some`. Si está por debajo (riesgo de colisión con árboles) o por encima (violación del espacio aéreo), debe devolver `None`. El programa debe empezar pidiendo la altura del dron. El usuario la introduce y el programa debe contestar con el mensaje adecuado y solicitar de nuevo la altura del dron.
- 💡 Pistas para el alumno:
 - Recuerda usar los operadores lógicos `&&` para evaluar si el número está dentro del rango permitido.
 - En el `main`, evalúa el resultado de la función usando un bloque `match` para imprimir alertas personalizadas en caso de recibir `None`.

🔧 Prueba de fuego: Si el usuario introduce 5, el programa debe decir «Alerta: Dron fuera de rango seguro». Si introduce 150, debe confirmar «Altitud estable a 150 metros».

7.2. Ejercicio 2: El Validador de Contraseñas del Sistema (Práctica con `Result<T, E>`)

- **Objetivo:** Gestionar errores informativos personalizados utilizando las variantes `Ok` y `Err`.
- **Enunciado:** Diseña una función `validar_password(password: &str) -> Result<(), String>`. Las reglas de seguridad del sistema exigen que la contraseña tenga como mínimo 8 caracteres. Si cumple la regla, devuelve `Ok()`. Si es más corta, devuelve un `Err` con el mensaje: «Seguridad insuficiente: la contraseña debe tener al menos 8 caracteres».
- 💡 Pistas para el alumno:
 - Puedes medir la longitud de un texto en Rust utilizando el método `.len()` sobre la cadena.
 - Como la función solo valida y no necesita transformar ningún dato, usa el tipo unidad `()` para la variante de éxito.
- 🔧 Prueba de fuego: Al probar con «12345», el programa no debe cerrarse con pánico; debe imprimir limpiamente el texto del error devuelto por la función.

7.3. Ejercicio 3: El Lector Defensivo de Archivos (Manejo de archivos y ?)

- **Objetivo:** Aprender a capturar los errores del sistema operativo de manera controlada sin que el programa colapse (`panic`).

- **Enunciado:** Escribe un programa que intente leer el contenido de un archivo llamado **secreto.txt** e imprimirlo por pantalla. Si el archivo no existe en la carpeta del proyecto, el programa de Rust fallará por defecto. Tu misión es capturar ese error de **entrada/salida** (**std::io::Error**) usando el operador `?` en una función y, en el main, mostrar un mensaje amigable que diga: "Aviso: El archivo "secreto.txt" no se encuentra en el directorio actual. Por favor, créalo.".
-  **Pistas para el alumno:**
 - Estructura tu función para que devuelva un `Result<String, std::io::Error>`.
 - No crees el archivo `secreto.txt` al principio; el objetivo es que compruebes primero cómo reacciona tu código cuando el archivo no está.
-  **Prueba de fuego:** Ejecuta el programa. Si se interrumpe bruscamente con un mensaje de panic, la gestión de errores está mal hecha. Debe terminar de forma limpia mostrando tu aviso personalizado.

7.4. Ejercicio 4: El Procesador de Inventario (Parsing y operaciones matemáticas)

Objetivo: Extraer datos numéricos de un archivo de texto plano y procesarlos de forma matemática.

Enunciado: Imagina que tienes un archivo llamado **inventario.txt** donde cada línea contiene únicamente el stock de un producto (por ejemplo, la primera línea dice «45», la segunda «12», etc.). Escribe un programa que lea el archivo línea por línea, convierta (parsee) cada línea a un entero (u32) y calcule la suma total de artículos en el almacén.

-  **Pistas para el alumno:**
 - Puedes usar `std::fs::read_to_string` para obtener todo el texto y luego el método `.lines()` para separar el contenido fila por fila mediante un bucle `for`.
 - Recuerda que `.parse::<u32>()` devuelve un `Result`. Usa `?` o un `match` para extraer el número de forma segura.
-  **Prueba de fuego:** Crea un archivo **inventario.txt** con los números 10, 20 y 30 (uno por línea). El programa debe imprimir exactamente: «Total de artículos en inventario: 60».

7.5. Ejercicio 5: El Diario Personal Orientado a Registros (Uso de `openOptions`)

- **Objetivo:** Trabajar con la persistencia de datos acumulativa (modo append) sin destruir la información existente.
- **Enunciado:** Escribe una herramienta interactiva para la terminal que funcione como un diario de notas rápido. El programa debe pedirle al usuario que escriba una línea de texto por consola (sus pensamientos del día). Al pulsar Enter, el programa debe guardar esa línea en un archivo llamado **diario.txt**. Cada vez que el alumno ejecute el programa, la nueva nota debe escribirse en una línea nueva al final del archivo, conservando todo lo que se escribió en los días anteriores.
-  **Pistas para el alumno:**

- Investiga el uso de **`std::fs::OpenOptions`**. Necesitarás activar los métodos **`.append(true)`** y **`.create(true)`**.
- Utiliza la macro `writeln!` (con una “n” al final) en lugar de `write!` para asegurar que cada anotación del diario empiece en una línea limpia.
-  **Prueba de fuego:** Ejecuta el programa tres veces seguidas introduciendo textos diferentes. Al abrir el archivo `diario.txt` con cualquier editor de notas, deberías ver las tres frases guardadas perfectamente en tres líneas distintas.